

std::bind

std::bind

std::is_invocable

std::bind

std::result_of std::function

std::is_invocable

```
// SFINAE-unfriendly callable
struct sfinae_unfriendly {
    template <typename First, typename ...Rest>
    struct returns {
        static_assert(
            !std::is_floating_point_v<First>,
            "First argument can't be a floating point type");
        using type = First;
    };
};

template <typename ...Args>
auto operator()(Args&&... args) const
-> typename returns<Args...>::type
{ /*...*/ }
};

// SFINAE-friendly call wrapper
template <typename F>
class call_wrapper {
    F _f;
public:
```

```

call_wrapper(F f) : _f(std::move(f)) {}

template <typename ...Args>
auto operator()(Args&&... args) const // const prepends an int
-> std::invoke_result_t<F const&, int, Args...>
{ return _f(1, std::forward<Args>(args)...); }

template <typename ...Args>
auto operator()(Args&&... args) volatile // volatile prepends a double
-> std::invoke_result_t<F volatile&, double, Args...>
{ return _f(1.0, std::forward<Args>(args)...); }
};

// attempt to call const-qualified operator(),
// which calls sfinae_unfriendly with int argument
call_wrapper const w = sfinae_unfriendly{};
w(); // error: static assertion failed
// static_assert(!std::is_floating_point_v<First>)
// required by substitution of
// std::invoke_result_t<sfinae_unfriendly volatile&, double, Args...>
// call_wrapper<sfinae_unfriendly>::operator()<Args...>(Args&&...) volatile
// [with Args = {}]'

```

return

```

int i;
std::bind([](auto& x) -> void { x = 1; }, i)(); // fine
std::bind([](auto& x) { x = 1; }, i)(); // error: assignment of read-only reference 'x'
// required by substitution of
// Result std::_Bind<lambda(auto&), int>::operator()<Args..., Result>(Args&&...) const
// [with Args = {}]; Result = <missing>

```

i

```

// SFINAE-unfriendly call wrapper
template <typename F>
class call_wrapper {
    /*...*/

    template <typename ...Args>
    decltype(auto) operator()(Args&&... args) const // const prepends an int
    { return _f(1, std::forward<Args>(args)...); }

```

```

template <typename ...Args>
decltype(auto) operator()(Args&&... args) volatile // volatile prepends a double
{ return _f(1.0, std::forward<Args>(args)...); }
};

call_wrapper const w = sfinae_unfriendly{};
w(); // fine, calls const-qualified operator()

```

is_callable
not_fn

std::result_of

```

struct fun {
    template <typename ...Args>
    void operator()(Args&&... args) = delete;

    template <typename ...Args>
    bool operator()(Args&&... args) const { return true; }
};

std::bind(fun{})(()); // error: call to deleted function

```

std::not_fn

```

std::not_fn(fun{})(()); // fine, returns false

```

-
- std::bind std::bind<R>
- std::tuple_element

```

std::bind([](auto& x) -> void { x = 1; }, 42)(); // fine
std::bind([](auto& x) { x = 1; }, 42)(); // error

```

- result_type

```

std::bind([](auto& x) -> void { x = 1; }, 42)(); // fine
std::bind<void>([](auto& x) { x = 1; }, 42)(); // error

```

-
- `std::result_of`
- `is_callable`
- `not_fn`